

ANALYSIS

Orphaned-dependency sweep — 2026-07-28

Field	Value
Revision	1
Created	2026-07-28
Last modified	2026-07-28
Status	active
Trigger	§11.4.108(5) — three “fixed-but-not-working” discoveries in one cycle

Table of contents

- [Why this sweep exists](#)
- [The real architectural finding](#)
- [Detector 1 — per-commit compile integrity](#)
- [Detector 2 — stale comment premises](#)
- [Detector 3 — mutation protocols](#)
- [Detector 4 — untested guards](#)
- [Detector 5 — moved symbols in fixtures](#)
- [The gate](#)
- [Honest gaps](#)

Why this sweep exists

§11.4.108(5): “*≥ 3 ‘fixed-but-not-working’ discoveries in one cycle signal an architectural VERIFICATION flaw, NOT three independent bugs; on the 3rd, STOP patching symptoms and fix the VERIFICATION pipeline.*”

Three instances surfaced in one remediation cycle, all the same shape — **a change moves or renames something, and a dependent artefact elsewhere is left pointing at the old world:**

#	Instance	Closed by
1		34e264e1

#	Instance	Closed by
	respModel — a test's composite literal referenced a struct field that existed only in the author's working tree, never in the committed tree	
2	helix_agent — a refactor moved memoryMu's lock into beginMemoryWrite(); a sibling test's mutation protocol still named saveMemories()	5b88aea6
3	debate_orchestrator — a dedup fix reconciled the test-side comment but left the production comment asserting the fixed defect in present tense, and un-fed the guard's fixture	2a9485e1

Existing gates miss this class because **they check the CHANGED file, never the artefacts that DEPEND on what changed.**

The real architectural finding

The checker already existed. helix_code/Makefile:355 ships:

```
verify-compile-tests: ## Compile-check ALL test code under -tags=nogui (go
```

which runs `go test -tags=nogui -run='^$' -count=1 ./...` — this *does* compile test files and *would* have caught instance 1.

The gap was never a missing checker. It is that the checker is only ever bound to the WORKING TREE. An author whose tree carries an uncommitted symbol sees a truthfully-green gate and ships a commit that compiles for nobody else. The pasted ok 0.018s in instance 1 was not fabricated — it accurately described a tree that was never committed.

So the fix is binding the existing checker to each **commit**.

Detector 1 — per-commit compile integrity

All 14 commits of the meta-repo delta swept. **Exactly three non-compiling commits — the three already known. No new ones.**

FAIL: 3fd55a4d, 3c8197cf, 905a0b0a PASS: 8bdac5b0 (baseline), 614353fb, c7484cc7, 640db264, ed0cdcb8, 34e264e1, 97d5ad2b, 99ff7d8e, dc9e4caa, bf1ce692, c2b6945e

All three fail with a **byte-identical** error — 3c8197cf/905a0b0a fail by inheritance, not as independent defects:

```
internal/server/llm_generate_native_model_regression_test.go:53:3:
  unknown field respModel in struct literal of type modelRecordingProvider
FAIL    dev.helix.code/internal/server [build failed]
```

Key mechanism: respModel is a field on a *test-only* struct referenced from a *second test file*. go build never sees it. Only a test-compiling check works.

Detector 2 — stale comment premises

Two NEW findings, both HIGH.

helix_agent/internal/router/router.go:481-482 — comment claims “Emitting the identity field here closes that false-GREEN vector at the source.” The **next commit** 0367570e states verbatim that “*that claim was INCORRECT*” — and never touched router.go, so the refuted claim survives at HEAD. A maintainer could weaken checkHelixAgentHealth on that false premise, reopening the mock-matching false-GREEN vector.

Stale claims in the §11.4.131 handoff docs. bf1ce692 genuinely repaired the F1 RED branch, but c2b6945e — dispatched to correct the status — changed only ~6 lines per document, leaving the claim alive at RESUME.md:55 (inside the paste-ready resumption prompt), :67, :260 (under “NOT yet fixed”), and docs/CONTINUATION.md:15. A fresh session was instructed to repair something already repaired. Closed by 25e54521.

Enumerated negatives: main-repo Go source **ZERO** across 10 commits / ~20 files; debate_orchestrator **ZERO**.

Detector 3 — mutation protocols

Two NEW, low severity.

LOW — `helix_agent/internal/services/protocol_monitor_cpu_percent_test.go` cites lock calls at `protocol_monitor.go` lines 183/245/255/265/294/683. All six checked: **none contains a lock** (183 = func signature, 245 = return result, 255/265 = field assignments, 294 = `}`), 683 = `strconv.ParseInt`). The `RED_MODE` instruction itself is accurate; only the line citations drifted.

COSMETIC — `debate_log_repository_select_priority_test.go`: 46 cites line “252-ish”, actual 275; guard string verified present.

Harvest: `helix_code` 39 paired mutation / 61 \$1.1 / 764 `RED_MODE` lines; `helix_agent` 13/21/215; `debate_orchestrator` 3/5/47. Every *target-naming* protocol verified correct.

Detector 4 — untested guards

HIGH — `helix_agent/.../ollama/ollama.go`: 69a5ab8d added ctx-guards at **four** send sites; the dedicated test drives **only** the `httpClient.Do` site. The other three are reached by no test — deleting them fails zero tests. The meta-repo closed the identical gap for `vertexai/groq/bedrock/ensemble` in 97d5ad2b; **ollama was left behind** — the same reconciliation-in-one-place pattern.

HIGH — `helix_agent/internal/agents/dream/dreamer.go`: 934-957 **atomicWriteFile**: grep of the whole dream test corpus for `atomicWriteFile / loadMemories / orientationPhase` returns **zero hits**. The concurrency tests assert only `memoryWritersPeak`, unaffected by write atomicity. Reverting to `os.WriteFile` fails zero tests.

MEDIUM — **the three applications/*/main_racefix_test.go**: correctly-shaped tests existed but all carry `//go:build !nogui`, and `verify-compile-tests` runs `-tags=nogui` — so they were never even compiled. **Superseded**: a follow-on investigation found Fyne’s ci build tag runs them headlessly; all nine now execute and pass, and doing so exposed 137 real data races. See the sibling document `docs/research/gui_racefix_execution_gap_20260728/ANALYSIS.md`.

Detector 5 — moved symbols in fixtures

EMPTY. Every removed func/type/const/var across 24 commits enumerated and grepped at HEAD. Only the known `respModel` instance. Specifically clean: the ctx-parameter signature changes in 905a0b0a have zero stale call sites; `runCleanupWorkerRaceHelper` has zero remaining references.

The gate

`scripts/gates/commit_compile_integrity_gate.sh` — binds the existing `verify-compile-tests` checker to each COMMIT rather than the working tree. Modes: `default @{u}..HEAD`, `--range`, `--last N`, `--advisory`, `--tags`, `--self-test`. Throwaway detached worktrees only, live submodules/ symlinked in, skips commits touching no inner Go, EXIT/INT/TERM trap removes every worktree and prunes.

§1.1 paired-mutation self-test — the load-bearing artefact:

expect DOES-NOT-COMPILE on known-bad 3fd55a4d:

```
3fd55a4d DOES-NOT-COMPILE 61s
```

```
internal/server/llm_generate_native_model_regression_test.go:53:3:
```

```
unknown field respModel in struct literal of type modelRecordingPr
```

```
=> PASS: gate correctly FAILED the known-bad commit
```

expect COMPILES on known-good 34e264e1:

```
34e264e1 COMPILES 50s
```

```
=> PASS: gate correctly PASSED the known-good commit
```

SELF-TEST PASS: gate can both fail and pass — §1.1 pairing intact

Measured cost (honest): ~50 s per compile-relevant commit warm-cache; ~20 s when failing fast. Full 14-commit sweep ~11 min, ~790% CPU, peak RSS ~940 MB, ~0 disk. Cheap enough for pre-push over a delta; **too expensive for verify-all-constitution-rules.sh** (~50 s × N on a frequently-run sweep). **Deliberately NOT wired into the aggregator** — that is a separate reviewable decision with a real cost budget.

What it does NOT cover: Detectors 1 and 5 only. Detectors 2/3/4 are not type-checkable. Two bounded follow-ons, both suggested by remediations already in-tree: CM-MUTATION-PROTOCOL-TARGET-EXISTS (assert symbols named in mutation prose still exist) and CM-DEFENCE-IN-DEPTH-CITES-TEST (require a guard comment to name a test, assert it exists — exactly what 2a9485e1 did by hand; mechanising it would have flagged both HIGH Detector-4 findings).

Honest gaps

- **!nogui GUI sources are compile-unverified by this gate on this host.** Exactly 3 first-party packages sit behind that boundary. Without `-tags=nogui` the run fails on missing X11/GL headers alone; reporting that as FAIL would be a §11.4.201 FAIL-bluff, hence the pinned tag.
- **The sweep holds submodules/ at live working-tree state**, not each commit's pinned SHAs — it proves main-repo delta integrity, not main×submodule pin-pair integrity.

- **helix_agent / debate_orchestrator** were NOT compile-swept (outside Detector-1 scope). helix_agent's delta is **pushed** — if any commit does not compile, that is live in a published submodule. Follow-on dispatched.
- **Detector-4 verdicts rest on reading call sites, not executed mutation.**
“Deleting this fails zero tests” is a strong read-based inference, not a captured mutation run.
- **HEAD moved four times mid-sweep**
(bf1ce692 → c2b6945e → 683205eb → 5300a4e6); Detectors 2-4 were gathered against a moving tree.

Structural observation

The same shape recurred at **four layers in one cycle** — source, test prose, production comment, and the session-handoff documents themselves. The unifying signature is *a reconciliation applied in one place and not its siblings*.

2a9485e1 demonstrates the correct remediation: fix the premise **and** name the test that holds it.